

GSDD Assignment report

Miguel Lourenço 70374
Rodrigo Cebola 70457

Topic

Our project consisted of a WebGIS that allows visualization, filtering and analysis of road accidents data from all over the UK. The application allows users to explore accident locations and their attributes, like severity and other conditions like the weather at the time of the accident or the type of road where it happened.

Its primary focus is an interactive map interface where users can do spatial analysis, checking for high risk areas or understand how different conditions affect the accident severity. This map is accompanied by a statistics sidebar, that allows for more in-depth analysis.

We used 2 datasets:

- [Road accidents dataset](#) - Contains point based data, with coordinates and other attributes;
- [UK districts dataset](#) - Contains the geometry of all UK administrative districts.

Compulsory functionalities

- A: Accident points and district geometries are stored on a PostgreSQL database and visualized using Leaflet.
- B: When selected, each accident displays its attributes in the statistics sidebar.
- C: The dynamic legend was accomplished by the map legend and the statistics sidebar. When in the normal markers map, the legend shows what the color of the markers mean, and on the choropleth, what the color of the districts mean. Furthermore, when filtering the accidents, the statistics sidebar shows information for all the currently visible accidents, as well as a count.
- D: The sidebar on the right allows users to filter the accidents based on their severity, the weather conditions and the days of the week. The backend queries the database dynamically using these attributes.
- E: These are by default handled by Leaflet.
- F: Next to the filters button, there exists a full extent button that zooms the map so that all of the currently visible accidents are on screen. On the choropleth map, the button also works when a district is selected, and zooms to that district's accidents. When no districts are selected, it just zooms out so that the whole island of the UK is visible.
- G: Spatial analysis was achieved in various ways, all using Turf:
- The color of the choropleth map comes from how many accidents occur on each district. To get the accident count, the backend checks the number of accidents that are contained in the district boundaries. Because the accidents are numerous, and there are 374 districts, checking every district for every accident would be too slow. To speed this up, we used a RBush spatial index, which is an efficient spatial index for storing points and rectangles. For each district, we store the bounding box. Then, for each accident, we ask the index what bounding boxes it is inside. This usually returns only 1-3 districts. Only those districts are used to check if the accident is inside their boundaries or not.
 - The spatial filter filters the accidents that are within a certain radius of a selected location. The radius of the filter circle is variable.
 - When selecting a district in the choropleth map, the accidents that are inside that districts' boundaries are revealed, when normally no individual accident markers are shown.

We also used the leaflet marker cluster plugin, for reducing visual clutter when the map is very zoomed out, and the leaflet heatmap plugin.

Optional functionalities

- H: When right clicking on the map, the user can add a new accident on that location. When clicking on an accident, the accident details are shown on the statistics sidebar. At the bottom, the user can enter the update mode, where changes can be made to the location of the accident and other attributes. The user can also delete the accident in this sidebar. All these changes are made to the persistent database.
- I: Accident data can be bulk imported with the bulk import feature. The accident data must be a csv file, and have the same headers as the original data's csv file. This feature acts as a way to add accident data for following years, for example. The newly inserted data is immediately shown on the map, and counts for the statistics. It also immediately counts for the choropleth colors for its districts.

Additional functionalities

We developed a sidebar that contains statistics related to the data that is currently being shown. These statistics consist of charts of the data points' attributes, more specifically a severity pie chart, a casualty histogram, and a line chart of the number of accidents per week day. These statistics are updated dynamically, depending on the filters that are applied.

The application has a heatmap layer, as well as a choropleth layer, where each district is color coded based on the number of accidents, and the user can select a district to filter accidents by where they happened. This filtering is achieved by clicking on the desired district. The accidents are then shown with markers. The statistics also reflect the accidents of the selected district.

User Interface design

Below are screenshots of how the interface looks:

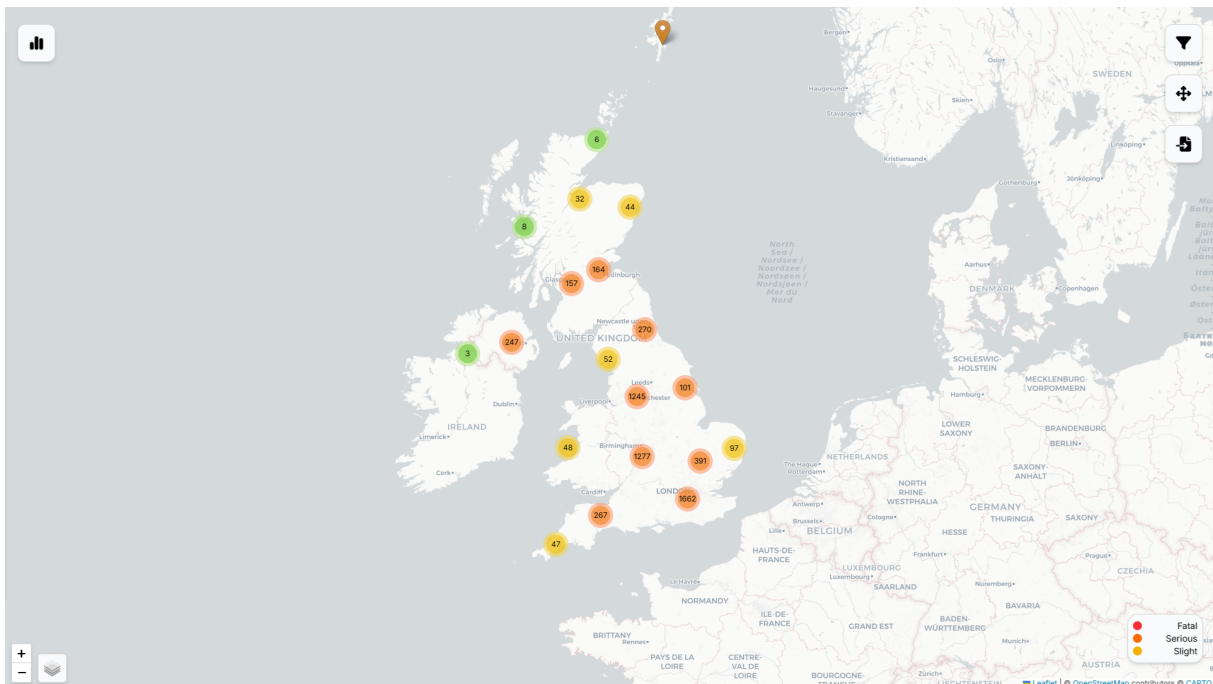


Figure 1: Accident clustering when zoomed out.

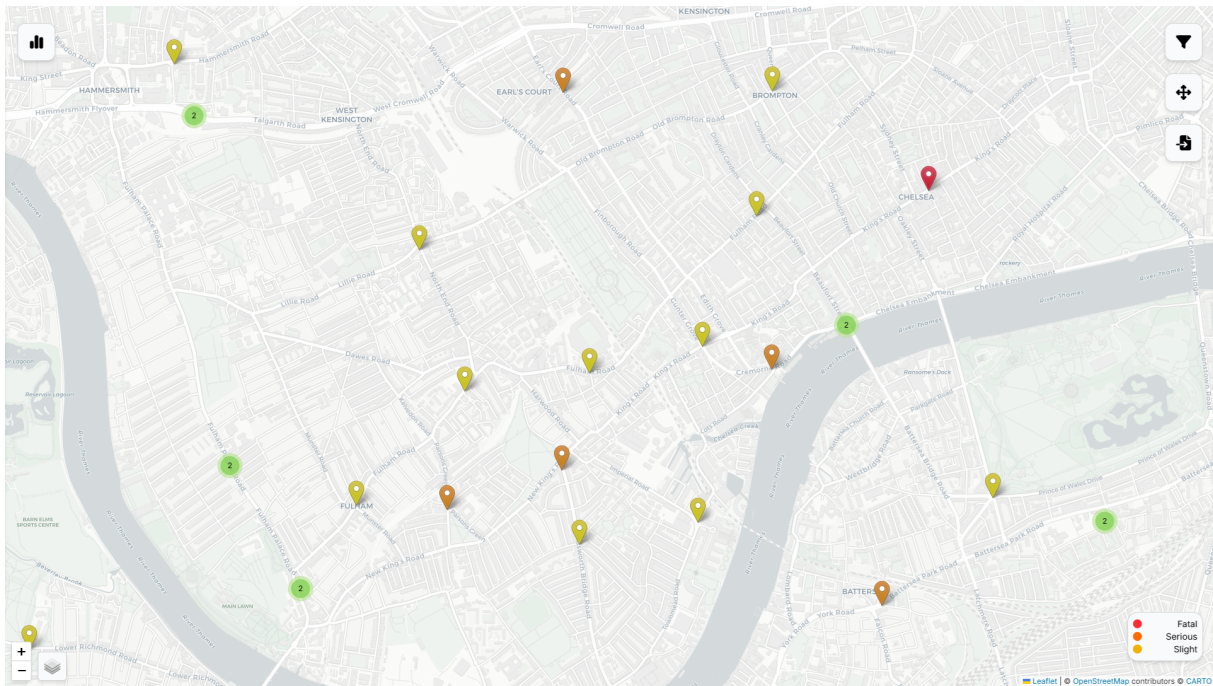


Figure 2: Map with zoom. Accidents are color coded based on their severity.

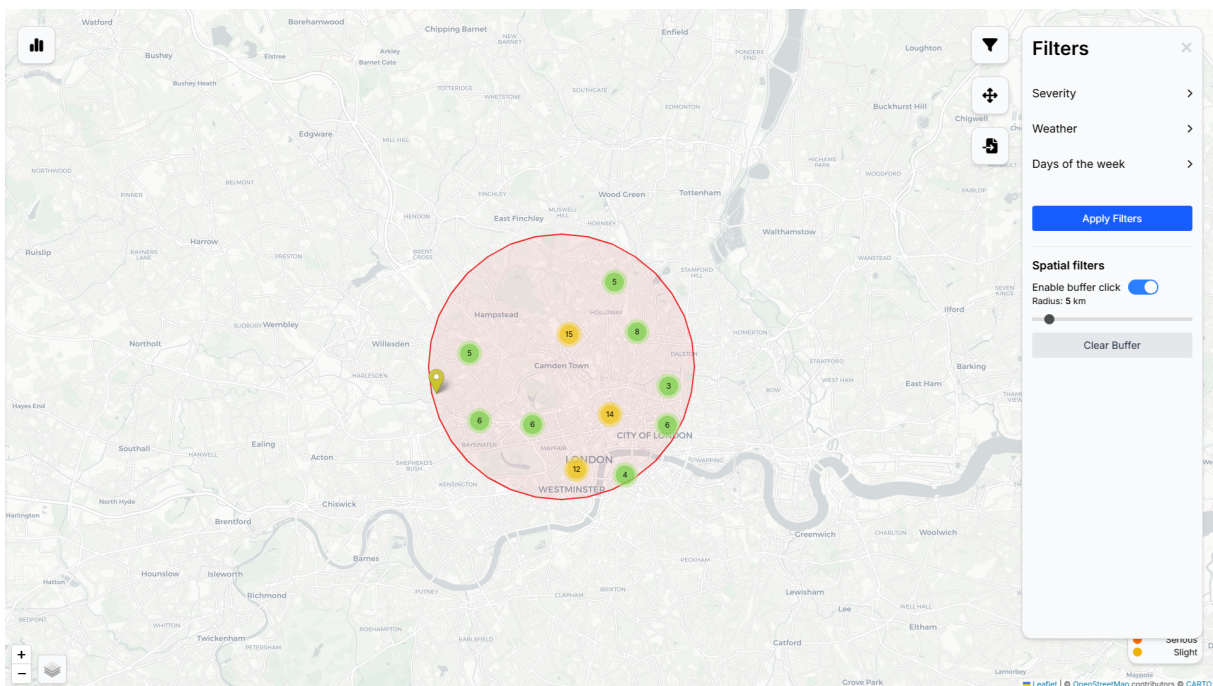


Figure 3: Map filtering. Filtering on 5 km radius of selected location (Camden Town).

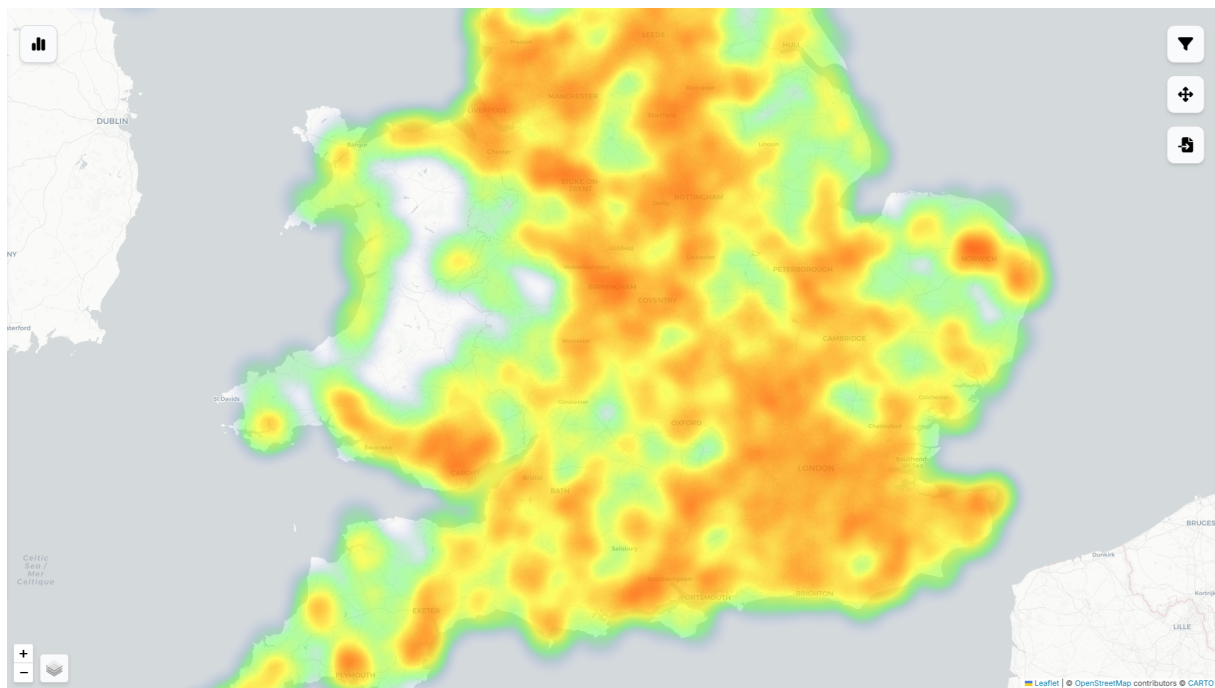


Figure 4: Accident heatmap.

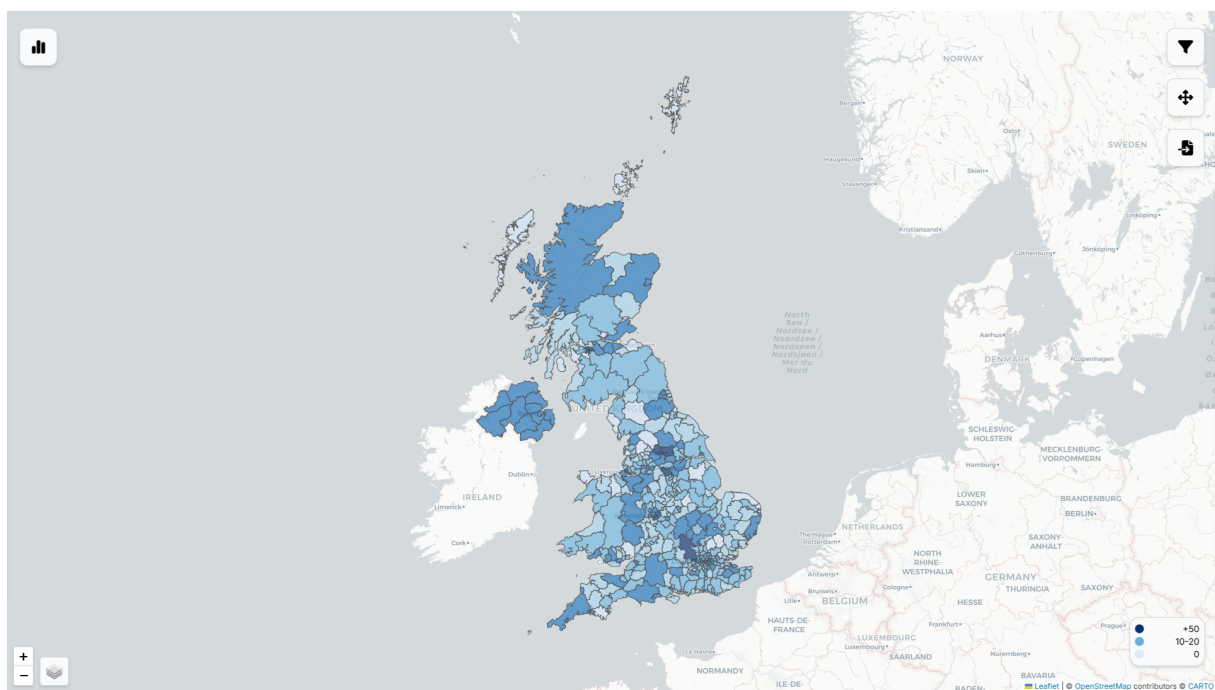


Figure 5: Choropleth map of districts.

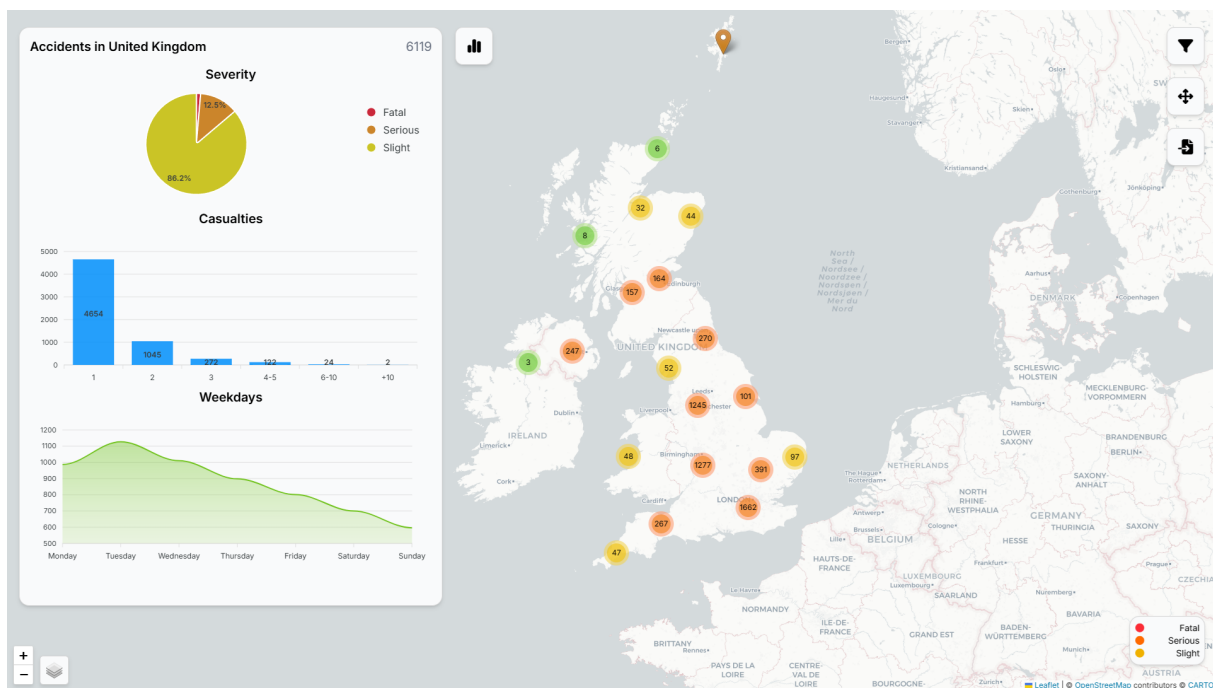


Figure 6: Statistics sidebar for the whole UK.

Database structure

Our database is a simple PostgreSQL + PostGIS database with only 2 tables. These are the **accidents** table, where all the information about the accidents is stored, and the **districts** table, where all the districts geometry is stored. Although PostGIS is enabled in the database, most of the spatial functionalities were achieved by the use of Turf, with the data on memory and not with spatial queries.

Architecture and technologies

For the front-end, we used HTML, Tailwind CSS, JavaScript, Leaflet, Turf and Apex Charts. For the backend, we used Node and express, to create the APIs. For the database, we used a PostgreSQL + PostGIS database like mentioned before.

The leaflet front-end communicates with the Node backend via Rest APIs, running locally. Then, the back-end queries the database for the filtered data, returning the data to the front-end for visualization and further calculations or spatial filtering.

Use of AI tools

AI tools were used to create skeletons of some front-end parts. We described what we wanted to show, and used the response as a base, and then further enhanced it to match the style of the rest of the application.

They were also used for code debugging, and for creating some SQL queries, like the insert of accidents and bulk insert.

Additional comments

The application runs locally, with the use of a node server and vite front end. As far as we know, there are no bugs.

Due to the enormous nature of our dataset, the current implementation only shows accidents from the first two weeks of 2021. This is to reduce the number of markers shown, which not only clutter the map, but also cause lag. This is hard-coded and can't be changed, but a time selection feature could be added to change this.